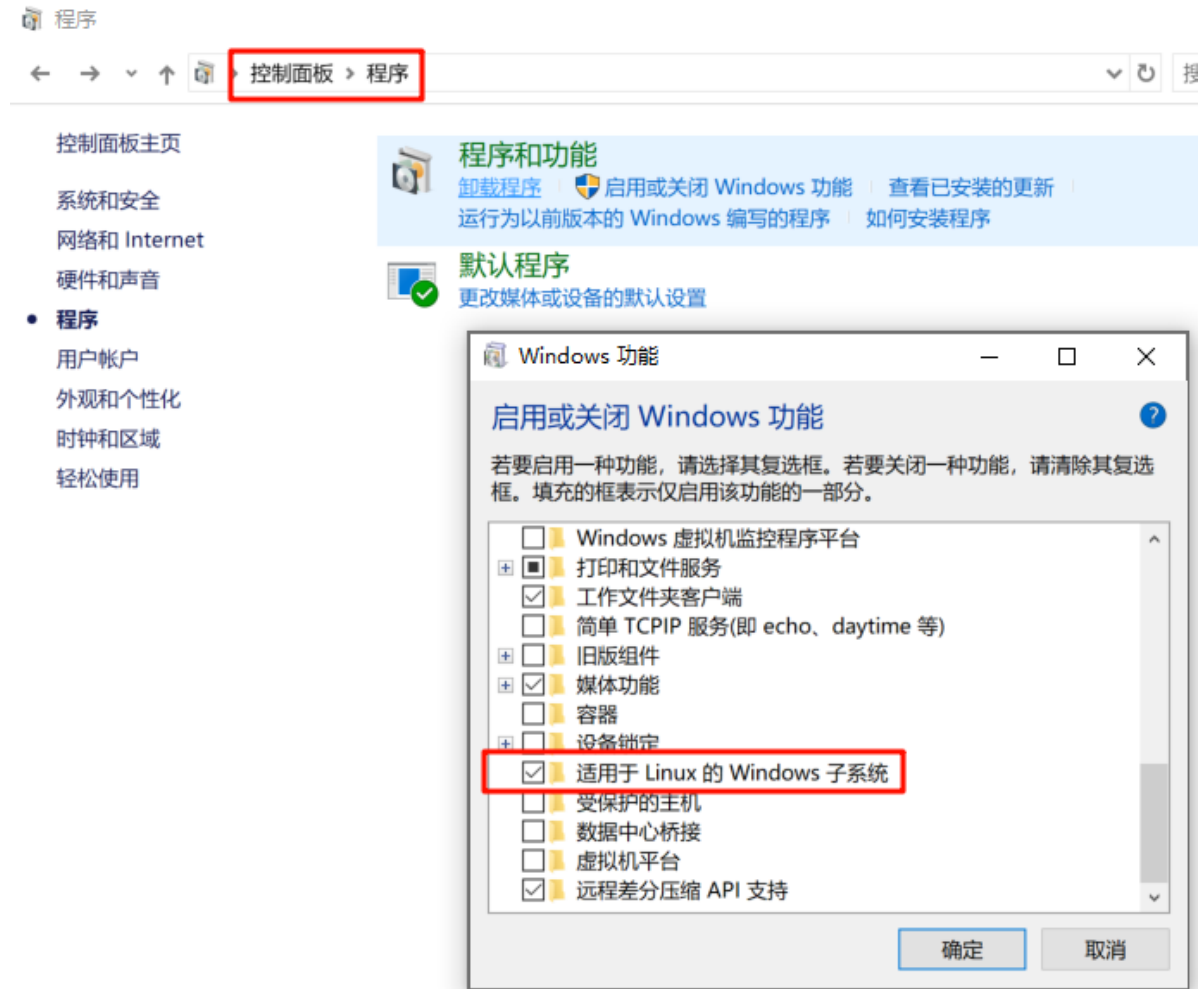


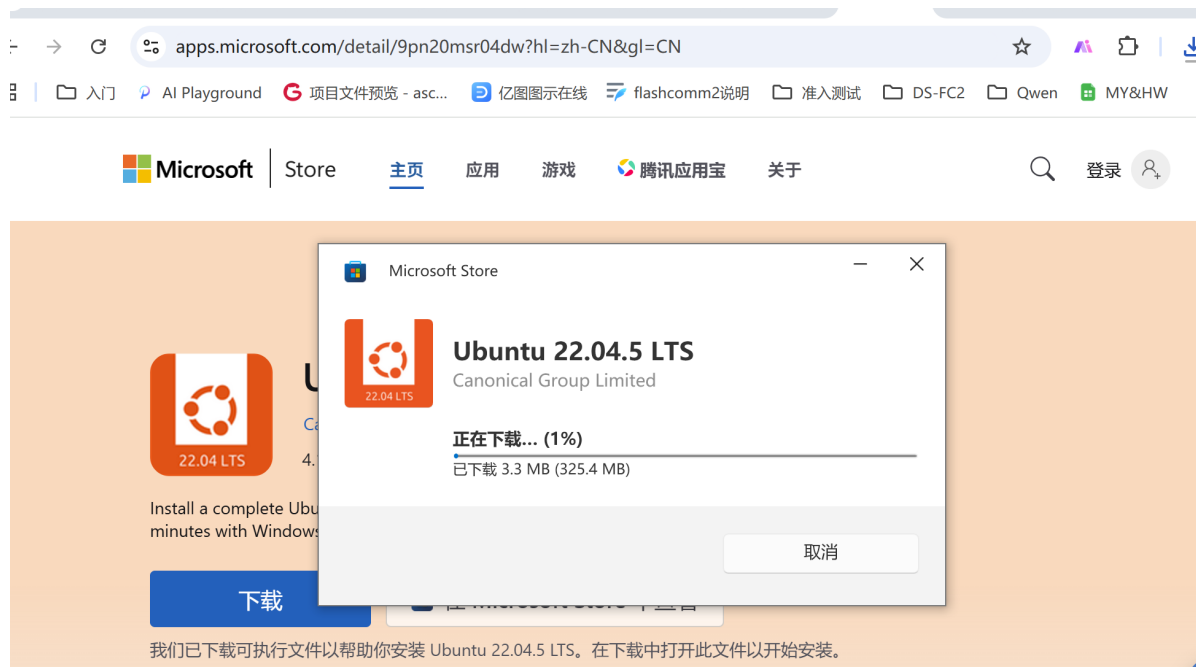
测试环境构建

安装wsl

- 打开设置



- 重启
- 获取Ubuntu22.04



迁移到D盘

#导出发行版

```
wsl --export Ubuntu-22.04 D:\wsl-ubuntu22.04.tar
```

#注销原发行版

```
wsl --unregister Ubuntu-22.04
```

#导入到 D 盘

```
mkdir D:\wsl-ubuntu
```

```
wsl --import Ubuntu-22.04 D:\wsl-ubuntu D:\wsl-ubuntu22.04.tar --version 2
```

换源

<https://www.linuxmi.com/ubuntu-22-04-apt-sources-list.html>

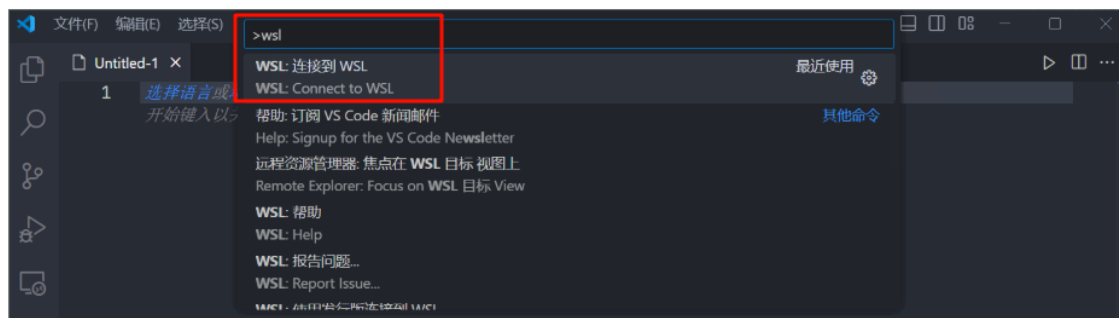
vscode连接wsl

安装wsl插件后连接即可

<https://blog.csdn.net/yanbober/article/details/138245581>

2. 在 VSCode 中打开 WSL 的项目。

通过使用 VSCode 快捷方式 CTRL+SHIFT+P 调出命令面板，键入 WSL，你将看到可用的选项列表，你可以在 WSL 会话中重新打开文件夹等。



git sshkey

剪切板

copyQ

- 下载

<https://github.com/hluk/CopyQ/releases>

- 教程

<https://zhuanlan.zhihu.com/p/662085789>

命令行工具

太好了！这是一个非常棒的问题。将 **Zsh**、**Oh My Zsh** 和 **zsh-autosuggestions** 这三者组合起来，会形成一个“1+1+1 > 3”的超级终端环境。它们各自扮演不同的角色，共同为你提供一个极其强大且优雅的命令行体验。

安装

在 Ubuntu 22.04 上配置 Zsh 及其生态工具可以极大地提升你的终端使用体验。下面是一份详细的安装指南和效率提升技巧。

1. 完整安装步骤

1.1 安装 Zsh

首先更新软件包列表并安装 Zsh：

```
sudo apt update
sudo apt install -y zsh
```

安装完成后，通过以下命令验证安装是否成功：`zsh --version`。

接下来将 Zsh 设置为默认 Shell：

```
chsh -s $(which zsh)
```

执行此命令后，系统会提示你输入当前用户的密码以授权更改。

1.2 安装 Oh My Zsh

Oh My Zsh 是一个社区驱动的框架，用于管理 Zsh 配置。

使用官方脚本安装：

```
sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

或者使用 wget：

```
sh -c "$(wget
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh -O -)"
```

如果在国内网络环境，可以使用 Gitee 镜像加速安装。

安装过程中会询问是否将 Zsh 设置为默认 Shell，输入 `y` 确认。安装完成后，最好完全注销并重新登录系统，以确保更改生效。

1.3 安装 zsh-autosuggestions 插件

zsh-autosuggestions 是一个智能提示插件，能根据历史命令提供建议。

克隆插件到 Oh My Zsh 自定义插件目录：

```
git clone https://github.com/zsh-users/zsh-autosuggestions ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-autosuggestions
```

启用插件，编辑 `~/.zshrc` 文件：

```
nano ~/.zshrc
```

找到 `plugins` 行，添加 `zsh-autosuggestions`：

```
plugins=(git zsh-autosuggestions)
```

保存文件后，重新加载配置：`source ~/.zshrc`。

2. 常用功能与效率提升技巧

2.1 实用快捷键大全

熟练掌握快捷键能极大提升命令行效率。

行编辑快捷键：

- `Ctrl + A`：移动到行首
- `Ctrl + E`：移动到行尾
- `Ctrl + U`：删除到行首
- `Ctrl + K`：删除到行尾
- `Ctrl + W`：删除前一个单词
- `Ctrl + Y`：粘贴已删除的内容

历史命令操作：

- `Ctrl + R`：反向搜索历史命令
- `Ctrl + P`：上一条命令
- `Ctrl + N`：下一条命令

zsh-autosuggestions 专用：

- `→` 或 `End` 键：接受当前建议
- `Ctrl + F`：接受整个建议（部分配置）

2.2 主题配置与个性化

Oh My Zsh 提供了丰富的主题选择。修改 `~/.zshrc` 文件中的 `ZSH_THEME` 变量来更换主题：

```
ZSH_THEME="mikeh" # 流行主题之一
```

或者使用随机主题: `ZSH_THEME="random"`。

高级用户可以考虑安装 powerlevel10k 主题:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git
${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
```

然后在 `.zshrc` 中设置: `ZSH_THEME="powerlevel10k/powerlevel10k"`。

2.3 增强插件配置

除了 `zsh-autosuggestions`, 还可以安装语法高亮插件:

```
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git ${ZSH_CUSTOM:-
~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting
```

然后在 `.zshrc` 的 `plugins` 数组中添加: `plugins=(git zsh-autosuggestions zsh-syntax-highlighting)`。

自定义 `zsh-autosuggestions` 行为:

- 修改建议颜色: `ZSH_AUTOSUGGEST_HIGHLIGHT_STYLE="fg=cyan"`
- 设置建议策略: `ZSH_AUTOSUGGEST_STRATEGY=(history completion)`
- 自定义快捷键接受建议: `bindkey '^ ' autosuggest-accept` (设置 `Ctrl+Space` 接受建议)

2.4 Git 快捷键集成

Oh My Zsh 的 Git 插件提供了大量实用别名:

- `gst = git status`
- `ga = git add`
- `gc = git commit`
- `gp = git push`
- `gl = git pull`
- `gco = git checkout`
- `gcm = git checkout master`

2.5 智能补全与纠正

启用自动纠正功能, 在 `~/.zshrc` 中添加:

```
setopt correct
```

这样当输入错误命令时, `zsh` 会提示纠正建议。

3. 故障排除与优化

3.1 解决常见问题

新终端窗口仍使用 `Bash`

运行 `echo $SHELL` 检查当前 Shell。如果显示 `/bin/bash`, 可能需要手动设置:

```
chsh -s $(which zsh)
```

然后完全注销并重新登录。

插件不生效

1. 确保插件目录路径正确
2. 检查 `.zshrc` 中插件名称拼写正确
3. 重新加载配置: `source ~/.zshrc`
4. 确认插件已正确克隆到自定义插件目录

命令提示颜色不明显

可以调整 `zsh-autosuggestions` 的显示颜色:

```
ZSH_AUTOSUGGEST_HIGHLIGHT_STYLE="fg=yellow"
```

3.2 性能优化建议

- 限制 `zsh-autosuggestions` 缓冲区大小以提高性能: `ZSH_AUTOSUGGEST_BUFFER_MAX_SIZE=20`
- 选择性启用插件, 避免加载不必要的插件
- 定期更新 Oh My Zsh 和插件: `omz update`

通过以上配置, 你的 Ubuntu 22.04 终端将变得更加强大和高效。 `zsh-autosuggestions` 的智能提示功能结合 Oh My Zsh 丰富的插件生态系统, 将显著提升命令行工作效率。

使用

下面我们分别介绍这三个工具, 并重点阐述它们组合后带来的核心便利功能及使用方法。

核心角色分工

工具	角色比喻	核心职责
Zsh	强大的发动机	替代系统默认的Bash, 提供更先进的脚本功能、可编程补全、强大的文件名通配等底层能力。
Oh My Zsh	贴心的改装套件 & 管家	基于Zsh, 提供海量开箱即用的 主题和插件 , 管理配置变得非常简单, 让Zsh变得既强大又易用。
zsh-autosuggestions	智能副驾驶	根据你的命令历史, 在你输入时给出灰色的自动完成建议, 大幅减少重复输入。

组合后的核心便利功能及使用指南

安装并配置好这三者后, 你将获得以下令人惊喜的功能:

1. 😎 炫酷且信息丰富的提示符

- **功能:** 通过 Oh My Zsh 的主题, 你的命令行提示符不再是简单的 `$` 或 `#`, 而是可以显示丰富的信息。
- **实现:** 修改 `~/.zshrc` 文件中的 `ZSH_THEME` 变量。例如, 非常流行的 `agnoster` 主题可以显示:
当前用户和主机名 **当前目录**的清晰路径 **Git仓库信息** (分支名、是否有未提交的更改 `*`、是否比远程领先 `↑` 或落后 `↓`) **上一条命令的返回状态** (如果失败, 提示符会变颜色或显示错误符号)
- **使用方法:** 编辑 `~/.zshrc`: `nano ~/.zshrc` 找到 `ZSH_THEME="robbyrussell"` (默认主题), 将其改为 `ZSH_THEME="agnoster"`。执行 `source ~/.zshrc` 立即生效。 *注意: 某些主题*

(如agnoster) 需要安装Powerline字体才能正确显示特殊符号。在iTerm2或终端设置中更换字体即可。

2. 智能命令补全与建议

这是最提升效率的功能之一，由 Zsh 和 zsh-autosuggestions 共同实现。

- **功能：Tab补全增强 (Zsh提供)：** 输入命令、文件名、路径甚至命令选项 (如 `git --` 后按Tab) 时，按 `Tab` 键会列出所有候选项，而不是Bash那样盲目地循环。你可以用方向键选择。 **自动建议 (zsh-autosuggestions提供)：** 当你输入一个曾经输入过的命令的开头时，它会立刻在后方以灰色文字给出完整的建议。
- **使用方法：** 输入 `git comm`，如果之前输入过 `git commit -m "message"`，灰色的建议就会自动出现。 **接受建议：** 直接按 `→` (右键) 或 `Ctrl + E` (移动到行尾)，整个建议就会被采纳。 **部分接受：** 按 `Alt + F` (向前移动一个单词) 可以逐词接受。

3. 海量插件生态系统

Oh My Zsh 的真正威力在于其插件。启用插件就像为你的终端安装“APP”。

插件名	功能描述	使用方法举例
git	最重要的插件之一。 为几乎所有常用的git命令创建了简短的别名。	- <code>gco main = git checkout main</code> - <code>gst = git status</code> - <code>ga . = git add .</code> - <code>gcmsg "fix" = git commit -m "fix"</code> - <code>gl = git pull</code> - <code>gp = git push</code>
z	目录快速跳转。 它会学习你经常访问的目录，让你用最短的输入跳转。	- 经常去 <code>/usr/local/nginx/conf</code> ，只需输入 <code>z nginx</code> 甚至 <code>z ng</code> 即可跳转。
sudo	快速添加sudo。	- 输入 <code>apt update</code> 后，按 <code>Esc</code> 两次，命令会自动变成 <code>sudo apt update</code> 。
copydir / copyfile	快速复制路径。	- 输入 <code>copydir</code> 复制当前目录的绝对路径。 - 输入 <code>copyfile file.txt</code> 复制文件内容到剪贴板。

启用插件方法： 编辑 `~/.zshrc`，找到 `plugins=(...)` 一行，添加插件名即可。例如：

```
plugins=(
  git
  z
  zsh-autosuggestions
  sudo
  copydir
  copyfile
)
```

保存后执行 `source ~/.zshrc`。

4. ⚡ 强大的历史命令搜索

- **功能：**不再需要反复按“上箭头”来查找历史命令。可以基于输入的内容进行智能搜索。
- **使用方法：**按 `Ctrl + R`，然后输入你记得的命令中的任意关键词（如 `docker`），Zsh 会实时过滤并显示匹配的历史命令。继续按 `Ctrl + R` 可以在匹配结果中向前循环。找到后按 `Enter` 执行，按 `→` 则可以将其带入命令行进行编辑。

5. 🔑 通配符和全局别名

- **功能：**Zsh 的文件通配功能更强大。
- **使用方法：**`ls **/*.txt`：递归列出当前目录及所有子目录下的 `.txt` 文件。`ls *.png` 即使没有 `png` 文件也不会报错，而是直接不展开。

常用快捷键一览

功能	快捷键	说明
移动到行首/尾	<code>Ctrl + A</code> / <code>Ctrl + E</code>	比按 <code>Home/End</code> 键更通用。
按单词移动	<code>Alt + ←</code> / <code>Alt + →</code>	快速在单词间跳转。
删除到行首	<code>Ctrl + U</code>	清空当前输入。
删除前一个单词	<code>Ctrl + W</code>	
接受自动建议	<code>→</code> 或 <code>Ctrl + E</code>	接受 <code>zsh-autosuggestions</code> 的建议。
历史命令搜索	<code>Ctrl + R</code>	

总结

安装这套组合后，你的终端将焕然一新：

- **眼睛舒服了：**漂亮的主题和清晰的 Git 状态提示。
- **手指轻松了：**git 别名、自动建议、快速跳转让你输入更少的字符。
- **脑子省事了：**智能补全和历史搜索让你无需记忆冗长的命令和路径。

这套工具组合是专业开发者和命令行爱好者的标配，能极大提升日常工作效率和愉悦感。花一点时间熟悉它们，绝对是值得的投资。

python版本管理

使用 pyenv 进行 Python 版本管理详细指南

<https://docs.pingcode.com/ask/1007532.html>

`pyenv` 是一个强大的 Python 版本管理工具，支持多版本共存和灵活切换，特别适合开发需要不同 Python 版本的项目。

一、安装 pyenv

macOS / Linux

安装依赖:

```
`# Ubuntu/Debian
sudo apt-get install -y make build-essential libssl-dev zlib1g-dev \ libbz2-dev
libreadline-dev libsqlite3-dev wget curl llvm \ libncursesw5-dev xz-utils tk-dev
libxml2-dev libxmlsec1-dev libffi-dev liblzma-dev`
```

通过安装脚本安装: `curl https://pyenv.run | bash`

配置 Shell (以 Bash 为例) :

```
#在 `~/.bashrc`或 `~/.zshrc`末尾添加:
export PATH="$HOME/.pyenv/bin:$PATH"
eval "$(pyenv init --path)"
eval "$(pyenv virtualenv-init -)"
#然后执行:
source ~/.bashrc # 或 source ~/.zshrc`
```

二、基本命令

命令	说明
<code>pyenv install --list</code>	查看所有可安装的 Python 版本
<code>pyenv install 3.10.6</code>	安装指定版本 (如 3.10.6)
<code>pyenv uninstall 3.9.1</code>	卸载指定版本
<code>pyenv versions</code>	查看已安装的所有版本 (* 表示当前激活版本)
<code>pyenv global 3.10.6</code>	设置全局默认版本
<code>pyenv local 3.9.1</code>	在当前目录设置局部版本 (生成 <code>.python-version</code> 文件)
<code>pyenv shell 3.8.12</code>	在当前 Shell 会话临时使用指定版本

三、版本切换优先级

优先级从高到低:

1. `pyenv shell` → 会话级
2. `pyenv local` → 目录级 (通过 `.python-version` 文件)
3. `pyenv global` → 用户级
4. 系统默认 Python

四、虚拟环境管理 (pyenv-virtualenv)

1. **创建虚拟环境**: `pyenv virtualenv 3.10.6 myproject-env` # 基于 Python 3.10.6 创建
 2. **激活/停用环境**: `pyenv activate myproject-env` # 激活 `pyenv deactivate`
停用
 3. **删除虚拟环境**: `pyenv uninstall myproject-env`
-

五、高级用法

1. **指定镜像加速安装** (适用于国内用户) : `v=3.10.6; wget https://npm.taobao.org/mirrors/python/$v/Python-$v.tar.xz -P ~/.pyenv/cache/`
`pyenv install $v`
 2. **同时安装多个版本**: `pyenv install 3.9.1 pyenv install 3.11.0`
 3. **项目专用环境**: `cd myproject pyenv local 3.11.0` # 设置项目 Python 版本 `pyenv virtualenv 3.11.0 project-env` # 创建项目独立环境 `pyenv activate project-env pip install -r requirements.txt`
-

六、常见问题

1. **安装失败**: 检查依赖是否完整 (如缺失 `libffi-dev`)。使用 `pyenv install -v 3.x.x` 查看详细日志。
 2. **命令未找到**: 确认 Shell 配置已正确加载 (检查 `~/.bashrc` 或 `~/.zshrc`)。
 3. **恢复系统 Python**: `pyenv global system`
-

七、最佳实践

- **全局版本**: 设置为稳定的 LTS 版本 (如 3.10.x)。
- **项目环境**: 每个项目使用独立的 `virtualenv`。
- **版本锁定**: 在项目根目录使用 `.python-version` 文件确保协作一致性。

通过 `pyenv`, 你可以无缝管理从 Python 2.7 到最新版本的环境, 彻底解决多版本冲突问题!